



تمرین دو

برنامه‌سازی پیشرفته

دانشکده علوم مهندسی

طراح تمرین: ایلیا افضلی ، امیر حسینی ، سروش نصیری

بهار ۱۴۰۵



۱.۱ مقدمه و هدف تمرین

در این سوال با یکی از قدرتمندترین الگوریتم‌های حل مسئله آشنا خواهید شد: **Backtracking**. این الگوریتم با کاوش فضای حل به صورت بازگشتی، گزینه‌های ممکن را یک‌به‌یک امتحان می‌کند و در صورت بن‌بست، به عقب برمی‌گردد تا مسیر درست را بیابد. هدف این تمرین، پیاده‌سازی یک **حل‌کننده سودوکو** با استفاده از الگوریتم Backtracking است. تمرکز اصلی روی تفکر بازگشتی، مدل‌سازی شروط مسئله و استفاده صحیح از آرایه‌های دوبعدی و توابع بازگشتی است.

۱.۲ صورت مسئله

سودوکو یک پازل عددی است که روی یک جدول 9×9 تعریف می‌شود. این جدول به ۹ ناحیه 3×3 تقسیم شده است. هدف، پر کردن خانه‌های خالی (مقدار صفر) با ارقام ۱ تا ۹ است به‌گونه‌ای که:

- در هر **سطر**، هر رقم از ۱ تا ۹ دقیقاً یک بار ظاهر شود.
- در هر **ستون**، هر رقم از ۱ تا ۹ دقیقاً یک بار ظاهر شود.
- در هر **کدام** از ۹ ناحیه 3×3 ، هر رقم از ۱ تا ۹ دقیقاً یک بار ظاهر شود.

شما باید برنامه‌ای بنویسید که یک جدول سودوکوی ناقص را به عنوان ورودی دریافت کرده، آن را با الگوریتم Backtracking حل کند و جدول کامل‌شده را چاپ نماید. در صورتی که جدول ورودی هیچ راه‌حلی نداشته باشد، برنامه باید پیام مناسبی نمایش دهد.

۱.۳ مشخصات ورودی

برنامه شما باید به گونه‌ای طراحی شود که بتواند ورودی را دقیقاً بر اساس قالب زیر مدیریت کند:

۱. **جدول سودوکو:** ورودی شامل ۹ خط است. هر خط حاوی ۹ عدد صحیح است که با فاصله از هم جدا شده‌اند. مقدار ۰ نشان‌دهنده خانه‌ای خالی است که باید توسط برنامه پر شود. مقادیر ۱ تا ۹ نشان‌دهنده خانه‌های از پیش پر شده (سرنخ‌ها) هستند.

۲. **تضمین ورودی:** تضمین می‌شود که ورودی دقیقاً ۹ سطر با ۹ عدد در هر سطر خواهد بود.

مثال ورودی

```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```



۱.۴ مشخصات خروجی

برنامه شما باید خروجی را دقیقاً با فرمت زیر (حساس به فاصله‌ها) در کنسول چاپ کند. هرگونه چاپ پیام‌های اضافه باعث کسر نمره خواهد شد.

۱. **جدول حل‌شده:** در صورت وجود راه‌حل، جدول سودوکوی کامل‌شده به همان قالب ورودی (۹ سطر، هر سطر ۹ عدد با فاصله) چاپ می‌شود. یعنی هر کدام از ۹ سطر، به فرمت زیر خواهد بود.

```
<d1> <d2> <d3> <d4> <d5> <d6> <d7> <d8> <d9>
```

۲. **حالت بدون راه‌حل:** در صورتی که جدول ورودی هیچ راه‌حل معتبری نداشته باشد، دقیقاً عبارت زیر چاپ شود:

```
No solution exists
```

مثال خروجی (مطابق ورودی بالا)

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

⚠ توجه

- خانه‌هایی که در ورودی مقدار داشته‌اند (سرنخ‌ها) باید در خروجی دست‌نخورده باقی بمانند. برنامه تنها مجاز به تغییر خانه‌های خالی (صفر) است.
- از چاپ کردن هرگونه پیام اضافی (مانند "enter your table:") و غیره اکیداً خودداری کنید. از آنجایی که کد شما توسط سیستم داوری خودکار تست می‌شود، برنامه باید صرفاً جدول را به عنوان ورودی دریافت کرده و جدول حل شده را در خروجی چاپ کند.



۱.۵ الزامات پیاده‌سازی

برای کسب نمره در این تمرین، رعایت موارد زیر الزامی است:

1. الزام استفاده از **Backtracking**: الگوریتم اصلی حل سودوکو باید به صورت بازگشتی (Recursive) با رویکرد Backtracking پیاده‌سازی شود. استفاده از روش‌های غیربازگشتی یا کتابخانه‌های حل‌کننده مستقیم مجاز نیست.
2. تابع بررسی اعتبار: منطق بررسی اینکه آیا قرار دادن یک رقم در خانه‌ای مشخص معتبر است یا خیر، باید در یک تابع مستقل پیاده‌سازی شود. این تابع باید هر سه قید سطر، ستون و ناحیه 3×3 را به‌طور همزمان بررسی کند.
3. عدم تغییر ساختار سرنخ‌ها: الگوریتم باید از تغییر خانه‌هایی که در ورودی مقدار داشته‌اند (مقدار غیر از صفر) به‌طور کامل اجتناب کند. تنها خانه‌های خالی (مقدار صفر) مجاز به تغییر هستند.
4. مدیریت حالت بدون راه‌حل: برنامه باید حالتی را که جدول ورودی هیچ راه‌حل معتبری ندارد به‌درستی تشخیص داده و پیام `No solution exists` را چاپ کند.
5. ساختار و خوانایی کد: کل منطق برنامه نباید در تابع `main` نوشته شود. برنامه باید به توابع مجزا و معنادار تقسیم شده باشد. استفاده از آرایه دوبعدی `int board[9][9]` برای نمایش جدول الزامی است.

یادآوری: مفهوم Backtracking

الگوریتم Backtracking به این صورت عمل می‌کند: اولین خانه خالی را پیدا کن، ارقام ۱ تا ۹ را یک‌به‌یک امتحان کن. اگر رقمی معتبر بود، آن را قرار بده و به خانه بعدی برو (فراخوانی بازگشتی). اگر هیچ رقمی برای خانه‌ای معتبر نبود، خانه را خالی کن و به مرحله قبل برگرد (backtrack).



۱.۶ ساختار کد پیشنهادی

برای تمیزی کد، از شما انتظار می‌رود کل برنامه را در `main` ننویسید و منطق برنامه را به توابع مجزا بشکنید. پیاده‌سازی توابع زیر (یا مشابه آن‌ها) توصیه می‌شود:

```
// include directive
#include <iostream>

using namespace std;

// تابع دریافت جدول سودوکو از ورودی استاندارد
void readBoard(int board[9][9]);

// تابع چاپ جدول سودوکو در خروجی استاندارد
void printBoard(int board[9][9]);

// سه قید معتبر بودن سطر و ستون و ناحیه را
// برای عدد مورد نظر در یک خانه مشخص بررسی می‌کند
bool isValid(int board[9][9], int row, int col, int num);

// Backtracking تابع اصلی حل‌کننده با الگوریتم
// برمی‌گرداند false و در غیر این صورت true در صورت موفقیت
bool solve(int board[9][9]);

int main() {
    int board[9][9];
    readBoard(board);

    if (solve(board))
        printBoard(board);
    else
        cout << "No solution exists" << endl;

    return 0;
}
```



۲.۱ مقدمه و هدف تمرین

در این سوال، شما یک حل‌کننده آمیرزا (Anagram Solver) پیاده‌سازی خواهید کرد. بازی آمیرزا یکی از بازی‌های کلاسیک و محبوب در زمینه پردازش متن و الگوریتم‌های جستجو است.

۲.۲ صورت مسئله

آمیرزا یک بازی ساده و معروف است که در آن به وسیله‌ی مجموعه‌ای از حروف، بایستی کلمات معنا دار را حدس بزنیم. در این تمرین از شما می‌خواهیم که یک آمیرزا حل کن بسازید.

قضیه از این قرار است که ابتدا تعدادی کلمه انگلیسی معنا دار به برنامه داده می‌شود این کلمات دیکشنری شما را تشکیل می‌دهند، سپس در چند سری، حروف به هم ریخته به شما داده می‌شود. از برنامه‌ی شما انتظار می‌رود که در هر سری تمام کلمات ممکن را که معنا دارند، با حروف به هم ریخته بسازد.

۲.۳ مشخصات ورودی

ورودی شامل چهار بخش است:

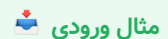
- خط اول شامل عدد n است که تعداد کلمات معنا دار در دیکشنری را مشخص می‌کند.
- در n خط بعدی، در هر خط یک کلمه‌ی معنا دار وارد می‌شود.



کلمات فقط از حروف کوچک تشکیل شده‌اند و از space در کلمات استفاده نشده است

- سپس عدد q وارد می‌شود.

- در q خط بعدی، در هر خط تعدادی حروف به هم ریخته وارد می‌شود که با space از یکدیگر جدا شده‌اند. (اینها کوئری‌های شما هستند)



مثال ورودی

```
3
amir
amin
arian
2
a i r s n m
n i a s p
```



۲.۴ مشخصات خروجی

برنامه شما باید به ازای هر سری حروف به‌هم‌ریخته (کوئری)، خروجی را به صورت زیر تولید کند:

- خط اول: تعداد کلمات قابل‌ساخت از حروف داده‌شده.
- خطوط بعدی: هر کلمه در یک خط جداگانه، به ترتیب لغت‌نامه‌ای (Alphabetical Order)

⚠ توجه

- خروجی کلمات هر کوئری باید حتماً بر اساس ترتیب لغت‌نامه‌ای مرتب شوند.
- در صورتی که ساختن هیچ کلمه‌ای ممکن نبود، تنها باید عدد 0 نمایش داده شود.
- هر کلمه فقط یک‌بار چاپ شود (حتی اگر به روش‌های مختلف قابل‌ساخت باشد)
- اگر یک حرف مثل `a` در کوئری فقط یک‌بار ظاهر شده بود نمیتوانید از آن دوبار استفاده کنید برای ساخت کلمه‌ای مانند `aram` (دقیقاً مانند بازی آمیرزا)

مثال خروجی 🚩 (بر اساس ورودی بالا):

```
2
amin
amir
0
```

۲.۵ الزامات پیاده‌سازی

برای کسب نمره در این تمرین، رعایت موارد زیر الزامی است:

- الزام استفاده از **Backtracking**: الگوریتم اصلی حل آمیرزا باید به صورت بازگشتی (Recursive) با رویکرد Backtracking پیاده‌سازی شود. استفاده از روش‌های غیربازگشتی یا کتابخانه‌های حل‌کننده مستقیم مجاز نیست.
- مدیریت حالت بدون جواب: برنامه باید حالتی را که کوئری ورودی هیچ راه‌حل معتبری ندارد به‌درستی تشخیص داده و پیام 0 را چاپ کند.
- ساختار و خوانایی کد: کل منطق برنامه نباید در تابع `main` نوشته شود. برنامه باید به توابع مجزا و معنادار تقسیم شده باشد.

📌 راهنمایی

کلمات معنا دار را به گونه‌ای ذخیره سازی کنید که امکان استفاده از الگوریتم backtraking به صورت بهینه وجود داشته باشد بر فرض در مثال بالا شما مجبور نیستید تمام مسیرها را بررسی کنید؛ مثلاً می‌دانیم هیچ کلمه‌ای با `ar` شروع نمی‌شود، پس تمام ترکیباتی که با `ar` آغاز می‌شوند حذف می‌شوند.



۲.۶ ساختار کد پیشنهادی

برای تمیزی کد، از شما انتظار می‌رود کل برنامه را در `main` ننویسید و منطق برنامه را به توابع مجزا بشکنید. پیاده‌سازی توابع زیر (یا مشابه آن‌ها) توصیه می‌شود:

```
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

// ورودی گرفتن دیکشنری
void loadDictionary(map<string, bool>& dictionary, int n);

// برای بکترک کردن
bool goForward(const string& word, const map<string, bool>& dictionary);

// پیدا کردن جایگشت های معتبر یک کوئری
vector<string> findValidCombinations(const vector<char>& letters, const map<string, bool>& dictionary);

// پردازش کوئری ها
void processQuery(const map<string, bool>& dictionary);

// اجرای بازی
void runGame();

int main() {
    runGame();
    return 0;
}
```




۳.۱ مقدمه و هدف تمرین

در این سوال شما مسئله ۸ وزیر با مانع را پیاده سازی خواهید کرد. تمرکز اصلی روی تفکر بازگشتی است.

۳.۲ صورت مسئله

یک صفحه شطرنج 8×8 داریم که تعدادی خانه‌ی مانع در آن مشخص شده است. شما باید تعداد حالات قرار گیری ۸ وزیر را روی صفحه حساب کنید به طوری که:

- هیچ دو وزیری یکدیگر را در سطر، ستون، یا قطرهای اصلی و فرعی تهدید نکنند.
- هیچ وزیری روی خانه‌های مانع قرار نگیرد.
- دقیقاً ۸ وزیر روی صفحه چیده شود (تعداد وزیرها ثابت و برابر با بعد صفحه است).

یادآوری

وزیر وقتی در یک خانه قرار گیرد تمام خانه های روی سطر و ستون و قطرهای آن را تهدید میکند.

۳.۳ مشخصات ورودی

ورودی برنامه دقیقاً مطابق قالب زیر از ورودی استاندارد خوانده می‌شود:

- ۸ خط، هر خط شامل ۸ کاراکتر (بدون فاصله) است.
- کاراکتر ☐ به معنی خانه آزاد (قابل قرار دادن وزیر)
- کاراکتر ☐ به معنی خانه مانع (قابل قرار دادن وزیر نیست)

تضمین ورودی:

- هر خط دقیقاً ۸ کاراکتر دارد (فقط شامل ☐ و ☐).

مثال ورودی

```
* .....  
.....  
.....  
.....  
.....*..  
.....  
.....  
.....*
```

خانه‌های (۱,۱)، (۵,۵) و (۸,۸) به عنوان مانع مشخص شده‌اند)



۳.۴ مشخصات خروجی

برنامه شما باید تنها یک عدد صحیح در خروجی چاپ کند که نشان‌دهنده تعداد کل چینش‌های معتبر قرار دادن ۸ وزیر روی صفحه ۸×۸ با در نظر گرفتن موانع است.

مثال خروجی (مطابق ورودی بالا)

76

توجه

- دقت کنید که موانع فقط برای جلوگیری از قرار دادن وزیر بر روی آنهاست. به این معنی که وقتی دو وزیر یکدیگر را تهدید میکنند و مانعی بینشان وجود دارد آن مانع از تهدید جلوگیری نخواهد کرد.
- خروجی برنامه فقط یک عدد صحیح است. هیچ پیام اضافی چاپ نشود.
- برنامه نباید خود چینش وزیرها را چاپ کند، فقط تعداد حالات را محاسبه و نمایش دهد.
- در صورتی که به دلیل موانع، هیچ چینش معتبری وجود نداشته باشد، عدد 0 چاپ می‌شود.

۳.۵ الزامات پیاده‌سازی

برای کسب نمره کامل در این تمرین، رعایت موارد زیر الزامی است:

- الزام استفاده از **Backtracking**: الگوریتم اصلی جستجو برای چیدمان وزیرها باید به صورت بازگشتی (Recursive) با رویکرد Backtracking پیاده‌سازی شود. استفاده از روش‌های غیربازگشتی (مثل حلقه‌های تو در تو با جایگشت) مجاز نیست.
- مدیریت موانع: خانه‌های مانع (*) باید در هر مرحله از جستجو غیرقابل انتخاب باشند. الگوریتم فقط خانه‌های آزاد (.) را امتحان می‌کند.
- ساختار و خوانایی کد: کل منطق برنامه نباید در تابع `main` نوشته شود. برنامه باید به توابع مجزا و معنادار تقسیم شده باشد. استفاده از آرایه دوبعدی `char board[8][8]` برای ذخیره صفحه (شامل . و *) الزامی است.

راهنمایی

در مسئله ۸ وزیر، به دلیل اینکه در هر سطر دقیقاً یک وزیر قرار می‌گیرد، معمولاً از یک آرایه یک‌بعدی `int queens[8]` استفاده می‌شود که در آن `queens[row] = col` یعنی وزیر سطر `row` در ستون `col` قرار دارد. این روش جستجو را بسیار بهینه می‌کند. سپس در هر سطر، ستون‌های ممکن را امتحان کرده و با کمک تابع `isSafe` بررسی می‌کنیم که آیا وزیر جدید با وزیرهای سطرهای قبلی در تضاد است یا روی مانع قرار می‌گیرد.



۳.۶ ساختار کد پیشنهادی

برای تمیزی کد، از شما انتظار می‌رود کل برنامه را در `main` ننویسید و منطق برنامه را به توابع مجزا بشکنید. پیاده‌سازی توابع زیر (یا مشابه آن‌ها) توصیه می‌شود:

```
#include <iostream>

using namespace std;

const int N = 8; // اندازه صفحه شطرنج (ثابت)

// تابع دریافت صفحه از ورودی
void readBoard(char board[N][N]);

// ستون row و تابع بررسی امن بودن قرار دادن وزیر در سطر col
bool isSafe(int queens[N], int row, int col, char board[N][N]);

// تابع حل
int solve(int queens[N], int row, char board[N][N]);

int main() {
    char board[N][N];
    readBoard(board);

    int queens[N] = {0}; // در ابتدا هیچ وزیری قرار نگرفته
    int totalSolutions = solve(queens, 0, board);

    cout << totalSolutions << endl;

    return 0;
}
```